Junior DevOps Engineer Exam

Linux, Containerization, CI/CD, High Availability

Overview

Welcome to this hands-on DevOps exam! In this challenge, you will demonstrate your ability to work with Linux systems, containerize applications, automate deployments, and manage production infrastructure. This exam tests your skills in:

- **Linux System Administration** - Essential command-line and system management skills
- **Docker Containerization** - Building production-ready container images
- **CI/CD Pipeline** - Automating build, test, and deployment workflows
- **High Availability Deployment** - Setting up fault-tolerant infrastructure

These skills are fundamental to modern DevOps practices and are essential for building reliable, scalable, and maintainable systems.

Time Allocation

Important Note: You have **1 week** to work on this exam. However, **you are not expected to rush through every requirement**. This is a comprehensive exam designed to assess your skills across multiple DevOps domains.

Your goal is to complete as much as you can within the 1 week, prioritizing:

- Quality over quantity
- Demonstrating your understanding of concepts
- Documenting your work clearly
- Following best practices

It is perfectly acceptable to submit partial work. Focus on doing a thorough job on the sections you choose to complete rather than rushing through everything.

Application Components

You will work with two sample applications:

- **API Backend**: https://bitbucket.org/metawhale/fast-api-clean/src/main/
- **UI Frontend**: https://bitbucket.org/metawhale/nextjs_app/src/main/

Technical Requirements

Prerequisites

- Linux environment (Ubuntu 20.04+ or similar distribution)
- Basic familiarity with command-line interfaces
- Docker and Docker Compose
- Git for version control
- Access to a CI/CD platform (GitHub Actions, GitLab CI, Jenkins, or Bitbucket Pipeline)
- Access to infrastructure for HA deployment (local VM, cloud platform, or container orchestration)

Repository Setup

- Clone or fork the sample applications to your preferred cloud repository platform (GitHub, GitLab, Bitbucket, etc.)

Expected Deliverables

You must create and provide:

1. **Linux Commands Documentation** - Screenshots or command outputs demonstrating Linux skills
2. **Dockerfile(s)** - Container definitions for both applications
3. **CI/CD Pipeline Configuration** - Pipeline definition file(s)
4. **Deployment Configuration** - HA setup configuration files
5. **Documentation** - README explaining setup and usage

Part 1: Linux Basics

Objective

Demonstrate fundamental Linux system administration skills that are essential for DevOps work. These skills form the foundation for container management, server administration, and troubleshooting production systems.

Requirements

Complete the following tasks and provide screenshots or command outputs as proof:

1. **File and Directory Management**
    - Create a directory structure: `/tmp/devops-exam/{app,logs,config,backup}`
    - Create an empty file named `app.log` in the `logs` directory
    - Copy a file from one directory to another
    - Move/rename a file
    - Create a symbolic link to a file
    - Find all `.log` files in a directory tree
    - Display the size of directories in human-readable format

2. **File Permissions and Ownership**
    - Change file permissions using numeric (chmod 755) and symbolic (chmod u+x) notation
    - Change file ownership (chown)
    - Create a file that only the owner can read and write
    - Make a shell script executable
    - Demonstrate understanding of permission bits (rwx for user, group, others)

3. **Text Processing and Searching**
    - Use `grep` to search for a pattern in files
    - Use `grep` with regular expressions (e.g., find lines starting with "ERROR")
    - Use `find` to locate files by name, size, or modification time
    - Use `cat`, `head`, `tail` to view file contents
    - Use `tail -f` to follow log files in real-time
    - Use `wc` to count lines, words, or characters
    - Use `sort` and `uniq` to process text data

4. **Process Management**

- List all running processes (`ps`, `top`, `htop`)
- Find processes by name using `ps aux | grep`
- Check system resource usage (CPU, memory, disk)
- Kill a process by PID
- Run a process in the background
- Bring a background process to foreground
- Monitor system processes in real-time

5. **Networking Basics**
   - Check network interfaces and IP addresses (`ip addr` or `ifconfig`)
   - Test network connectivity (`ping`)
   - Check open ports and listening services (`netstat` or `ss`)
   - Test if a specific port is open (`telnet`, `nc`, or `curl`)
   - Display DNS information (`nslookup` or `dig`)
   - Download files from the internet (`wget` or `curl`)
   - Check routing table (`ip route` or `route`)

6. **Package Management**
   - Update package lists (`apt update` or `yum update`)
   - Install a package (e.g., `nginx`, `htop`, `tree`)
   - Remove a package
   - Search for available packages
   - List installed packages

7. **System Information**
   - Display system information (`uname -a`)
   - Check disk usage (`df -h`)
   - Check disk usage for specific directories (`du -sh`)
   - Display memory usage (`free -h`)
   - Check system uptime and load average
   - View system logs (`journalctl` or `/var/log/`)

8. **User and Group Management**
   - Create a new user
   - Add user to a group
   - Switch to another user (`su`)
   - Execute commands as root (`sudo`)
   - Display current user information (`whoami`, `id`)

9. **Archiving and Compression**
   - Create a tar archive
   - Extract a tar archive
   - Compress files using gzip or bzip2

- Create and extract compressed archives (`.tar.gz`, `.tar.bz2`)
- Use `zip` and `unzip` commands

10. **Shell Scripting Basics**
    - Create a simple bash script with a shebang (`#!/bin/bash`)
    - Use variables in a script
    - Implement basic conditionals (if/else)
    - Use loops (for or while)
    - Make the script executable and run it
    - Redirect output to files (`>`, `>>`)
    - Use pipes to chain commands (`|`)

Example Tasks

Here are specific tasks you should complete:

Task 1: Log Analysis

```
# Create a sample log file with mixed content
# Search for all ERROR lines
# Count how many ERROR vs WARNING entries exist
# Find the 10 most recent log entries
# Extract unique IP addresses from access logs
```

Task 2: System Health Check Script

Create a bash script (`system_health.sh`) that:

- Displays current date and time
- Shows system uptime
- Displays CPU and memory usage
- Lists top 5 processes by CPU usage
- Shows available disk space
- Checks if specific services are running (e.g., docker, nginx)

Task 3: File Backup Automation

```
# Create a script that:
# - Creates a timestamped backup directory
# - Copies important files to backup directory
# - Compresses the backup into a .tar.gz file
# - Removes backups older than 7 days
```

Task 4: User and Permission Management

```
# Create a new user 'devops'
# Create a group 'developers'
# Add user to the group
# Create a shared directory with group write permissions
# Set up proper permissions for a web application directory
```

Deliverables

Provide a document (PDF, Markdown, or text file) with:

1. **Command History**: Screenshots or text showing the commands you executed
2. **Command Outputs**: The results of each command
3. **Explanations**: Brief explanation of what each command does
4. **Script Files**: Your bash scripts (system_health.sh, backup script, etc.)
5. **Screenshots**: Visual proof of successful command execution where appropriate

Implementation Guidelines

- **Documentation**: Clearly label each section and task
- **Real Examples**: Use real commands on an actual Linux system (VM, WSL, or native)
- **Understanding**: Show that you understand what each command does, not just copy-paste
- **Best Practices**: Follow Linux best practices and conventions
- **Safety**: Be careful with destructive commands (rm, kill, etc.)

Tips

- Use a Linux VM or WSL (Windows Subsystem for Linux) if you don't have a Linux system
- Practice in a safe environment before running commands on production systems
- Use `man <command>` or `<command> --help` to learn about commands
- Keep a command history for your documentation
- Test your scripts thoroughly before submitting
- Use `history` command to review your command history
- Consider using tools like `asciinema` to record terminal sessions

Part 2: Docker Containerization

Objective

Containerize the provided web applications using Docker to ensure consistent, reproducible environments across development and production.

Requirements

Create appropriate `Dockerfile(s)` for the applications with the following features:

1. **API Backend Containerization**
   - Create a `Dockerfile` for the FastAPI application
   - Set up the Python environment with required dependencies
   - Configure appropriate base image
   - Expose necessary ports
   - Use multi-stage builds for optimization (optional but recommended)

2. **UI Frontend Containerization**
   - Create a `Dockerfile` for the Next.js application
   - Set up the Node.js environment
   - Build the application for production
   - Configure appropriate base image
   - Expose necessary ports
   - Optimize for production deployment

3. **Local Execution**
   - Ensure both applications can be built using `docker build`
   - Ensure both applications can be run using `docker run`
   - Verify applications work correctly when containerized
   - Test inter-service communication if applicable

4. **Docker Compose (Optional but Recommended)**
   - Create a `docker-compose.yml` for local development
   - Configure both services
   - Set up networking between containers
   - Define environment variables

Example Commands

Your solution should support commands like:

```
# Build the API
docker build -t api-app:latest ./api

# Run the API
docker run -p 8000:8000 api-app:latest

# Build the UI
docker build -t ui-app:latest ./ui

# Run the UI
docker run -p 3000:3000 ui-app:latest

# Or using Docker Compose
docker-compose up --build
```

Implementation Guidelines

- **Base Images**: Choose appropriate base images (e.g., `python:3.10-slim`, `node:18-alpine`)
- **Layer Optimization**: Order Dockerfile commands to maximize cache efficiency
- **Security**: Don't run containers as root user
- **Environment Variables**: Use environment variables for configuration
- **Health Checks**: Implement Docker health checks (recommended)
- **.dockerignore**: Create `.dockerignore` files to exclude unnecessary files

Tips

- Start with the API backend first (typically simpler)
- Test each Dockerfile independently before combining
- Use multi-stage builds to reduce image size
- Document any environment variables required
- Consider using Docker Compose for easier local development

Part 3: CI/CD Pipeline

Objective

Create an automated Continuous Integration and Continuous Deployment pipeline that builds, tests, and deploys the applications automatically when code changes are pushed.

Requirements

Create a CI/CD pipeline configuration file that:

1. **Automatic Triggering**
   - Pipeline triggers automatically on commits to the `staging` branch
   - Support for manual triggers (optional)
   - Specify appropriate trigger conditions

2. **Build Stage**
   - Build Docker images for both applications
   - Tag images appropriately (e.g., with commit SHA, branch name)
   - Cache dependencies for faster builds (optional but recommended)
   - Validate Dockerfiles

3. **Test Stage**
   - Run automated tests for the applications
   - Execute linting/code quality checks
   - Run security scans on Docker images (optional but recommended)
   - Fail the pipeline if tests fail

4. **Deploy Stage**
   - Deploy to a staging environment
   - Push Docker images to a container registry (Docker Hub, ECR, GCR, etc.)
   - Update the running services with new images
   - Implement deployment strategies (rolling update, blue-green, etc.)

5. **Notifications**
   - Send notifications on pipeline success
   - Send notifications on pipeline failure
   - Include relevant details (commit info, logs, etc.)
   - Use email, Slack, Discord, or other notification services

Platform-Specific Examples

Choose one CI/CD platform and create the appropriate configuration:

GitHub Actions (`.github/workflows/deploy.yml`):

```
name: CI/CD Pipeline
on:
    push:
        branches: [staging]
jobs:
    build-test-deploy:
        runs-on: ubuntu-latest
        steps:
            - uses: actions/checkout@v3
            - name: Build Docker images
              run: docker-compose build
            # ... additional steps
```

GitLab CI (`.gitlab-ci.yml`):

```
stages:
    - build
    - test
    - deploy

build:
    stage: build
    script:
        - docker build -t api-app ./api
    # ... additional configuration
```

Bitbucket Pipeline (`bitbucket-pipelines.yml`):

```
pipelines:
    branches:
        staging:
            - step:
                  name: Build and Test
                  services:
                      - docker
                  script:
                      - docker build -t api-app ./api
```

Jenkins (`Jenkinsfile`):

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'docker build -t api-app ./api'
            }
        }
        // ... additional stages
    }
}
```

Implementation Guidelines

- **Pipeline as Code**: Store pipeline configuration in the repository
- **Environment Management**: Use separate configurations for staging/production
- **Secrets Management**: Use platform's secret management for credentials
- **Artifact Storage**: Push built images to a container registry
- **Rollback Strategy**: Have a plan to rollback failed deployments
- **Pipeline Stages**: Keep stages modular and focused

Tips

- Start with a simple pipeline (build → deploy) then add testing
- Use the platform's built-in Docker support
- Test pipeline changes in a separate branch first
- Configure notifications early for feedback
- Use caching to speed up pipeline execution

Part 4: High Availability Deployment

Objective

Deploy the applications in a high availability configuration to ensure fault tolerance, load distribution, and zero-downtime deployments.

Requirements

Deploy the applications with the following HA characteristics:

1. **Redundancy**
   - Deploy applications across at least 2 servers/VMs/nodes
   - Ensure automatic failover if one instance fails
   - Maintain service availability during node failures
   - Implement health checks for automatic recovery

2. **Load Distribution**
   - Distribute incoming requests across multiple instances
   - Implement load balancing (round-robin, least connections, etc.)
   - Ensure even distribution of traffic
   - Handle session persistence if required

3. **Domain-Based Access**
   - Configure applications to be accessible via domain names
   - Use DNS or hosts file configuration
   - No direct IP:port access required
   - Example: `http://api.myapp.local` and `http://ui.myapp.local`

4. **Container Orchestration (Choose One)**
   - **Kubernetes** (Highly Recommended) - Full orchestration with auto-scaling
   - **Docker Swarm** - Simpler orchestration for Docker containers
   - **VM-based** - Manual setup across multiple VMs with load balancer

Deployment Options

Option A: Kubernetes (Recommended)

Create Kubernetes manifests:

- **Deployments**: Define replica sets for both applications (min 2 replicas each)
- **Services**: Expose applications internally (ClusterIP)
- **Ingress**: Configure domain-based routing
- **ConfigMaps/Secrets**: Manage configuration and credentials
- **Health Probes**: Liveness and readiness checks

Example structure:

```
# api-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
    name: api-app
spec:
    replicas: 2
    # ... configuration
---
# api-service.yaml
apiVersion: v1
kind: Service
metadata:
    name: api-service
# ... configuration
---
# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
    name: app-ingress
# ... domain routing configuration
```

Option B: Docker Swarm

Create Docker Swarm configuration:

- **Stack file**: `docker-compose.yml` for swarm mode
- **Replicas**: Configure multiple replicas per service
- **Load balancing**: Use Swarm's built-in load balancer
- **Reverse proxy**: Traefik or Nginx for domain routing

```
version: "3.8"
services:
    api:
        image: api-app:latest
        deploy:
            replicas: 2
            restart_policy:
                condition: on-failure
        # ... configuration

    ui:
        image: ui-app:latest
        deploy:
            replicas: 2
        # ... configuration

    proxy:
        image: nginx:alpine
        # ... reverse proxy configuration
```

Option C: VM-Based Deployment

- Deploy applications on multiple VMs
- Configure Nginx/HAProxy as load balancer
- Set up health checks and failover
- Configure domain routing

Domain Configuration

Configure hosts file for local access:

```
# /etc/hosts (Linux/Mac) or C:\Windows\System32\drivers\etc\hosts
(Windows)
127.0.0.1 api.myapp.local
127.0.0.1 ui.myapp.local
```

Implementation Guidelines

- **High Availability**: Ensure no single point of failure
- **Monitoring**: Implement health checks and monitoring
- **Scaling**: Design for horizontal scaling
- **Configuration**: Use environment-based configuration
- **Networking**: Proper network isolation and security
- **Documentation**: Document architecture and access methods

Tips

- If new to Kubernetes, start with Minikube or kind for local testing
- Docker Swarm is simpler but less feature-rich than Kubernetes
- Test failover by manually stopping one instance
- Verify load distribution using logs or monitoring
- Document the architecture with diagrams if possible

Part 5: Solution Presentation

Objective

One week after receiving this exam, you will be required to present your completed work onsite.

Presentation Requirements

During the presentation, you should be prepared to explain:

- Your overall implementation approach
- The architecture of your solution
- Your thought process while solving each part
- Why you chose specific tools or configurations
- Challenges you encountered
- How you resolved those challenges
- Trade-offs or alternative solutions you considered
- How you would improve the solution with more time

You may also be asked to:

- Walk through your repository
- Explain your Dockerfiles
- Demonstrate your CI/CD pipeline
- Explain your High Availability deployment
- Answer technical questions about your implementation

Evaluation

The presentation is part of the assessment and will evaluate both your technical implementation.

Submission Guidelines

Submit whatever you have completed within the 24-hour timeframe. Partial submissions are expected and accepted.

Your submission should include:

1. **Linux Basics Documentation**
    - Document or PDF with command screenshots and outputs

- Bash scripts (system_health.sh, backup script, etc.)
- Explanations of commands used

2. **Dockerfiles**
   - `Dockerfile` for API backend
   - `Dockerfile` for UI frontend
   - `.dockerignore` files
   - `docker-compose.yml` (optional but recommended)

3. **CI/CD Configuration**
   - Pipeline configuration file(s) for your chosen platform
   - Any required scripts for build/deploy automation

4. **HA Deployment Configuration**
   - Kubernetes manifests (if using K8s)
   - Docker Swarm stack file (if using Swarm)
   - Load balancer configuration (if using VMs)
   - Network/infrastructure diagrams (optional)

5. **Documentation (README.md)**
   - Overview of your architecture
   - How to build and run containers locally
   - How to set up and test the CI/CD pipeline
   - How to deploy to HA environment
   - How to access applications via domain names
   - Troubleshooting guide
   - Architecture decisions and trade-offs

6. **Repository Links**
   - Links to your forked repositories
   - Link to CI/CD pipeline (if publicly accessible)

Note: We understand this is a time-limited exam. We're looking for depth of understanding in the areas you complete rather than superficial coverage of all areas.

Good Luck!

This exam will demonstrate your practical DevOps skills and your ability to build production-ready infrastructure. Take your time, document your decisions, and build something you'd be proud to show in a professional setting!